System Scaling Strategy — BFF, Kafka, Cache, Database & Beyond

Version: 1.0
Date: April 15, 2026
Author: Architecture Team
Parent Doc: banking-application-architecture.md

Table of Contents

1. Scaling Philosophy
2. Current Baseline & Growth Projections
3. BFF Layer Scaling
4. Kafka Scaling
5. Redis / Cache Layer Scaling
6. PostgreSQL Database Scaling
7. Elasticsearch Scaling
8. MongoDB Scaling
9. API Gateway Scaling
10. Microservice (Application Tier) Scaling
11. End-to-End Scaling Scenario: 10x Traffic
12. Scaling Bottleneck Analysis
13. Cost of Scaling
14. Scaling Runbook

---

1. Scaling Philosophy

1.1 Principles

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        SCALING PRINCIPLES                                 │
│                                                                           │
│  1. SCALE HORIZONTALLY FIRST                                              │
│     Add more pods/nodes/partitions before upgrading instance sizes.       │
│     Horizontal scaling has no theoretical ceiling. Vertical hits limits.  │
│                                                                           │
│  2. SCALE THE BOTTLENECK, NOT EVERYTHING                                  │
│     Profile first. Identify the actual bottleneck (DB? Kafka? CPU?        │
│     Network?). Scaling a non-bottleneck wastes money.                     │
│                                                                           │
│  3. AUTOSCALE WHERE POSSIBLE                                              │
│     Kubernetes HPA for stateless services. Kafka partition assignment     │
│     for consumers. Redis cluster resharding for memory.                   │
│                                                                           │
│  4. DECOUPLE TO SCALE INDEPENDENTLY                                       │
│     BFF scales independently from backend services.                       │
│     Read replicas scale independently from write primary.                 │
│     Kafka consumers scale independently from producers.                   │
│                                                                           │
│  5. CACHE TO AVOID SCALING                                                │
│     A 95% cache hit rate means only 5% of requests reach the DB.          │
│     Caching is the cheapest form of scaling.                              │
│                                                                           │
│  6. TEST SCALING BEFORE YOU NEED IT                                       │
│     Quarterly load tests (Gatling) simulate 10x traffic on staging.       │
│     Discover limits before production hits them.                          │
└──────────────────────────────────────────────────────────────────────────┘
```

1.2 Scaling Dimensions

Dimension	Mechanism	What It Scales
Horizontal (scale out)	More pods, nodes, partitions	Request throughput, consumer parallelism
Vertical (scale up)	Bigger instances	Per-node capacity (CPU, RAM, disk I/O)
Functional (decompose)	Split into more services	Isolate workloads (read vs write, search vs transactional)
Data partitioning	Sharding, table partitioning	Data volume, query parallelism
Geographic (scale wide)	Multi-region deployment	Latency, disaster recovery

2. Current Baseline & Growth Projections

2.1 Current Load Profile

Metric	Current (Day 1)	1 Year	3 Years
Active users	500K	2M	10M
Concurrent users (peak)	50K	200K	1M
Transactions/day	2M	10M	50M
Transactions/second (peak)	500	2,500	12,500
API requests/second (peak)	5,000	25,000	125,000
Kafka events/second	1,000	5,000	25,000
Transaction data (total)	50M rows	500M rows	3B rows
Redis memory usage	5 GB	20 GB	80 GB
Elasticsearch documents	50M	500M	3B

2.2 Scaling Tiers

```
TIER 1 (Day 1 — 500K users)       "Starter"
──────────────────────────

• 3-5 pods per service
• 3-node Kafka cluster (12 partitions for txn-events)
• 3-master Redis cluster (39GB total)
• PostgreSQL: 1 primary + 1 standby + 2 read replicas
• Elasticsearch: 3 data nodes

TIER 2 (Year 1 — 2M users)        "Growth"
──────────────────────────

• 5-10 pods per service (HPA active)
• 5-node Kafka cluster (24 partitions for txn-events)
• 6-master Redis cluster (78GB total)
• PostgreSQL: larger instance + 4 read replicas
• Elasticsearch: 6 data nodes + warm tier

TIER 3 (Year 3 — 10M users)       "Scale"
──────────────────────────

• 10-30 pods per service
• 9-node Kafka cluster (48 partitions for txn-events)
• 9-master Redis cluster (117GB total)
• PostgreSQL: largest instance + 6 read replicas + Citus for sharding
• Elasticsearch: 12 data nodes + warm + cold tier
```

---

3. BFF Layer Scaling

3.1 Why BFF Is a Scaling Hotspot

The BFF (Backend-for-Frontend) aggregates multiple downstream calls into a single client response. It handles every API request from the frontend, making it the highest-throughput service in the stack.

```
Client → API Gateway → BFF → { Account Service, Txn Service, Loan Service }

Dashboard load = 1 BFF call → 3 parallel downstream gRPC calls
Transaction list = 1 BFF call → 1 downstream call + filter assembly
Transfer = 1 BFF call → 1 downstream call + WebSocket push
```

3.2 BFF Scaling Strategy

```
┌──────────────────────────────────────────────────────────────────────┐
│                        BFF SCALING                                   │
│                                                                      │
│  Stateless ─── Horizontal Autoscale via Kubernetes HPA               │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  HPA Configuration:                                          │    │
│  │                                                              │    │
│  │  apiVersion: autoscaling/v2                                  │    │
│  │  metadata:                                                   │    │
│  │    name: bff-web-hpa                                         │    │
│  │  spec:                                                       │    │
│  │    scaleTargetRef:                                           │    │
│  │      name: bff-web                                           │    │
│  │    minReplicas: 3                                            │    │
│  │    maxReplicas: 30                                           │    │
│  │    metrics:                                                  │    │
│  │    - type: Resource                                          │    │
│  │      resource:                                               │    │
│  │        name: cpu                                             │    │
│  │        target:                                               │    │
│  │          type: Utilization                                   │    │
│  │          averageUtilization: 70                              │    │
│  │    - type: Pods                                              │    │
│  │      pods:                                                   │    │
│  │        metric:                                               │    │
│  │          name: http_requests_per_second                      │    │
│  │        target:                                               │    │
│  │          type: AverageValue                                  │    │
│  │          averageValue: 500                                   │    │
│  │    behavior:                                                 │    │
│  │      scaleUp:                                                │    │
│  │        stabilizationWindowSeconds: 30                        │    │
│  │        policies:                                             │    │
│  │        - type: Percent                                       │    │
│  │          value: 100      # double pods in one step           │    │
│  │          periodSeconds: 60                                   │    │
│  │      scaleDown:                                              │    │
│  │        stabilizationWindowSeconds: 300  # wait 5min before   │    │
│  │        policies:                        # scaling down       │    │
│  │        - type: Percent                                       │    │
│  │          value: 25       # remove 25% pods at a time         │    │
│  │          periodSeconds: 120                                  │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  Pod Resources:                                                      │
│  • Requests: 500m CPU, 512Mi memory                                  │
│  • Limits:   2 CPU, 1Gi memory                                       │
│  • JVM: -Xms256m -Xmx512m (Spring Boot + WebFlux for async I/O)     │
│                                                                      │
│  Scale triggers:                                                     │
│  • CPU > 70% → scale up                                              │
```

```
│   • Requests/sec > 500 per pod → scale up                                │
│   • Both below thresholds for 5 min → scale down (slowly)               │
└──────────────────────────────────────────────────────────────────────────┘
```

3.3 BFF Performance Optimizations at Scale

Optimization	Detail
Parallel downstream calls	Dashboard = `CompletableFuture.allOf(getBalance, getRecentTxns, getLoanEmi)`. 3 calls in parallel → total latency = max(individual latencies), not sum.
Connection pooling (gRPC)	gRPC channels are multiplexed (HTTP/2). Single connection carries hundreds of concurrent RPCs. No connection-per-request overhead.
Response compression	gzip for BFF-Web, Brotli for mobile. Reduces payload size 60-80%.
Circuit breaker per downstream	If Account Service is slow, Txn Service calls aren't affected (bulkhead isolation).
WebFlux (reactive)	BFF uses Spring WebFlux for non-blocking I/O. A single pod handles thousands of concurrent connections without thread-per-request.

3.4 BFF-Web vs BFF-Mobile Scaling Independently

```
BFF-Web:    Peak at 9 AM - 6 PM (working hours)     → Scale 3-15 pods
BFF-Mobile: Peak at 7 AM - 9 AM and 7 PM - 11 PM   → Scale 3-20 pods
(different traffic patterns → independent HPAs)
```

---

4. Kafka Scaling

4.1 Kafka Scaling Dimensions

```
┌──────────────────────────────────────────────────────────────────────┐
│                        KAFKA SCALING                                 │
│                                                                      │
│  Dimension           | How to Scale                | Impact          |
│  ────────────────────┼─────────────────────────────┼─────────────────┤
│  Write throughput    | Add brokers                 | More leaders =  |
│                      |                             | more write capacity|
│                      |                             |                 |
│  Consumer parallelism| Add partitions              | More partitions = |
│                      | (irreversible!)             | more consumers  |
│                      |                             | can work in     |
│                      |                             | parallel        |
│                      |                             |                 |
│  Consumer throughput| Add consumer instances      | Up to 1 consumer |
│                      | (up to #partitions)         | per partition max |
│                      |                             |                 |
│  Storage             | Add disk or reduce          | Retention period |
│                      | retention                   | vs storage cost |
│                      |                             |                 |
│  Durability          | Increase replication        | More copies =   |
│                      | factor                      | more fault tolerant|
│                      |                             | but more storage |
└──────────────────────────────────────────────────────────────────────┘
```

4.2 Partition Scaling Plan

Topic	Partitions (Tier 1)	Partitions (Tier 2)	Partitions (Tier 3)	Partition Key
`txn-events`	12	24	48	`accountId` (guarantees per-account ordering)
`account-events`	6	12	24	`accountId`
`notification-events`	6	12	24	`userId`
`audit-events`	6	12	24	`userId`
`fraud-events`	6	12	24	`accountId`

Why `accountId` as partition key? All events for a single account land on the same partition, guaranteeing ordering. A debit event is always processed before the credit notification for the same account.

4.3 Consumer Scaling

```
RULE: Number of active consumers ≤ Number of partitions

txn-events (12 partitions):
┌──────────────────────────────────────────────────────────────────┐
│  Consumer Group: notification-cg                                 │
│                                                                  │
│  Tier 1: 3 consumers (4 partitions each)                         │
│  ┌──────┐ ┌──────┐ ┌──────┐                                     │
│  │ C1   │ │ C2   │ │ C3   │                                     │
│  │P0,P1 │ │P4,P5 │ │P8,P9 │                                     │
│  │P2,P3 │ │P6,P7 │ │P10,P11│                                    │
│  └──────┘ └──────┘ └──────┘                                     │
│                                                                  │
│  Tier 2: 6 consumers (2 partitions each)                         │
│  ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐        │
│  │ C1   │ │ C2   │ │ C3   │ │ C4   │ │ C5   │ │ C6   │        │
│  │P0,P1 │ │P2,P3 │ │P4,P5 │ │P6,P7 │ │P8,P9 │ │P10,11│       │
│  └──────┘ └──────┘ └──────┘ └──────┘ └──────┘ └──────┘        │
│                                                                  │
│  Tier 3: 12 consumers (1 partition each) — maximum parallelism   │
│  If more throughput needed → increase partitions to 24           │
└──────────────────────────────────────────────────────────────────┘
```

4.4 Kafka Broker Scaling

Tier	Brokers	Instance	Disk	Justification
Tier 1	3	kafka.m5.xlarge (4 vCPU, 16GB)	1TB EBS gp3	Handles 1,000 events/sec with replication factor 3
Tier 2	5	kafka.m5.2xlarge (8 vCPU, 32GB)	2TB EBS gp3	More leaders = distribute write load. 5,000 events/sec.
Tier 3	9	kafka.m5.4xlarge (16 vCPU, 64GB)	4TB EBS gp3	25,000 events/sec. 3 brokers per AZ.

4.5 Key Kafka Scaling Metrics

Metric	Alert Threshold	Action
Consumer lag	> 10,000 messages for 5 min	Scale up consumers (add pods)
Consumer lag	> 100,000 messages for 5 min	Add partitions + consumers
Broker disk usage	> 75%	Reduce retention OR add brokers
Under-replicated partitions	> 0 for 5 min	Broker issue — investigate immediately
Request rate per broker	> 10,000 req/sec	Add brokers to spread leaders
Produce latency (p99)	> 100ms	Network or disk I/O bottleneck on broker

---

5. Redis / Cache Layer Scaling

5.1 Redis Cluster Scaling Path

```
Tier 1 (39GB):
┌─────────┐  ┌─────────┐  ┌─────────┐
│Master 1│  │Master 2│  │Master 3│     3 masters × 13GB = 39GB
│Replica1│  │Replica2│  │Replica3│     Handles: 500K users
└─────────┘  └─────────┘  └─────────┘

Tier 2 (78GB):
┌─────────┐ ┌─────────┐ ┌─────────┐ ┌─────────┐ ┌─────────┐ ┌─────────┐
│Master 1│ │Master 2│ │Master 3│ │Master 4│ │Master 5│ │Master 6│
│Replica1│ │Replica2│ │Replica3│ │Replica4│ │Replica5│ │Replica6│
└─────────┘ └─────────┘ └─────────┘ └─────────┘ └─────────┘ └─────────┘
6 masters × 13GB = 78GB  |  Online resharding - zero downtime
Handles: 2M users

Tier 3 (117GB+):
9 masters × 13GB = 117GB  |  OR upgrade to cache.r6g.2xlarge (52GB/node)
Handles: 10M users
```

5.2 Scaling Triggers

Trigger	Scaling Action
Memory > 70% on any master	Add shard (online resharding). Redis Cluster redistributes hash slots.
Connections > 80% of max	Add replicas (read traffic offloaded) or add shards.
Latency p99 > 2ms	Investigate: likely network saturation or oversized keys. Consider adding shards.
Evictions > 0	Memory pressure. Immediate: add shard. Short-term: review TTLs, identify large keys.
Hit rate drops below 90%	Not a cluster scaling issue — check TTLs, key patterns, application logic.

5.3 Scaling Without Downtime

Redis Cluster supports online resharding:

1. Add new master node to the cluster.
2. Redis automatically migrates a portion of hash slots from existing masters to the new master.
3. Clients are redirected via `MOVED` / `ASK` responses during migration.
4. Add a replica for the new master (different AZ).
5. Total migration time: minutes to hours depending on data volume. Zero downtime.

---

6. PostgreSQL Database Scaling

6.1 The Hardest Thing to Scale

PostgreSQL is the single most difficult component to scale because:

- It's stateful (data must be consistent).
- Writes must go to one primary (no multi-master in vanilla PG).
- Vertical scaling has a ceiling (~128 vCPU, 1TB RAM on largest RDS instances).

6.2 PostgreSQL Scaling Strategy (Layered)

```
┌──────────────────────────────────────────────────────────────────────┐
│                    POSTGRESQL SCALING LAYERS                          │
│                                                                       │
│  LAYER 1: Optimize Before Scaling (Cheapest)                          │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  • Index optimization (composite, partial, covering indexes) │    │
│  │  • Query optimization (EXPLAIN ANALYZE, fix N+1 queries)     │    │
│  │  • Connection pooling (PgBouncer: 300 app → 50 PG conns)    │    │
│  │  • Vacuum tuning (autovacuum_max_workers, naptime)           │    │
│  │  • Partition pruning (ensure date filters are always present)│    │
│  │  Cost: $0 (engineering time only)                            │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  LAYER 2: Read Replicas (Moderate Cost)                               │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  • Route read-only queries to replicas (@Transactional(RO))  │    │
│  │  • Tier 1: 2 replicas  |  Tier 2: 4  |  Tier 3: 6           │    │
│  │  • Dedicated replicas for: filters, reports, analytics       │    │
│  │  • Read replica latency: < 100ms (async replication)         │    │
│  │  Cost: ~$800/month per replica                               │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  LAYER 3: Vertical Scale (Expensive)                                  │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  • Upgrade RDS instance: db.r6g.2xlarge → 4xlarge → 8xlarge  │    │
│  │  • More CPU, RAM, I/O throughput for the write primary       │    │
│  │  • Ceiling: db.r6g.16xlarge (64 vCPU, 512GB RAM)             │    │
│  │  Cost: $1,600 → $3,200 → $6,400 → $12,800/month              │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  LAYER 4: Table Partitioning (Already Done)                           │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  • transactions table: PARTITION BY RANGE (created_at)       │    │
│  │  • Monthly partitions: transactions_2026_01, _02, ...        │    │
│  │  • Benefits: partition pruning, parallel scans, faster VACUUM │    │
│  │  • Old partitions detached and archived to cold storage      │    │
│  │  Cost: $0 (part of schema design)                            │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  LAYER 5: Horizontal Sharding (Last Resort — Tier 3+)                 │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  • Citus extension (distributed PostgreSQL)                  │    │
│  │  • Shard key: account_id                                     │    │
│  │  • Each shard = a PostgreSQL node owning a subset of accounts│    │
│  │  • Cross-shard queries (joins across accounts) become complex │    │
│  │  • Only when single-primary write throughput is exceeded      │    │
│  │  Cost: Significant engineering + operational complexity       │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  LAYER 6: CQRS Separation (Tier 2+)                                   │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  • Writes → primary PostgreSQL (normalized, ACID)            │    │
```

```
│  │  • Reads → denormalized read models (materialized views,      │    │
│  │    Elasticsearch, dedicated read databases)                   │    │
│  │  • Kafka events propagate write-side changes to read-side     │    │
│  │  Cost: Moderate (event consumers + read store infra)          │    │
│  └──────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
```

6.3 Read Replica Routing

```java
@Configuration
public class DataSourceConfig {

    @Bean
    @Primary
    public DataSource routingDataSource(
        DataSource primaryDs, DataSource replicaDs
    ) {
        var routing = new AbstractRoutingDataSource() {
            @Override
            protected Object determineCurrentLookupKey() {
                return TransactionSynchronizationManager.isCurrentTransactionReadOnly()
                    ? "replica" : "primary";
            }
        };
        routing.setTargetDataSources(Map.of("primary", primaryDs, "replica", replicaDs)
        routing.setDefaultTargetDataSource(primaryDs);
        return routing;
    }
}

// Usage: any @Transactional(readOnly = true) method routes to replica
@Transactional(readOnly = true)
public Page<TransactionDTO> listTransactions(TransactionFilterDTO filters, Pageable p)
    return transactionRepository.findAll(spec, p).map(mapper::toDTO);
}
```

6.4 Archival Strategy

```
Hot data (< 1 year):    Primary PG + Replicas (fast queries)
Warm data (1-3 years):  Detached partitions → separate PG instance (cheaper, slower)
Cold data (3-10 years): Exported to Parquet in S3 Glacier (compliance retention)
Delete (> 10 years):    Purged per retention policy
```

7. Elasticsearch Scaling

7.1 Scaling Dimensions

Dimension	How	When
Query throughput	Add replica shards	Search latency increases under load
Indexing throughput	Add data nodes	Indexing lag increases
Data volume	Add shards (via index rollover) or add data nodes	Shard size > 50GB
Memory (aggregations)	Increase JVM heap per node (up to 50% of RAM, max 31GB)	Aggregation queries OOM
Cost optimization	Warm/cold tiers (ILM)	Old indices rarely queried

7.2 Scaling Plan

Tier	Data Nodes	Shards per Monthly Index	Replicas	Capacity
Tier 1	3 (hot)	6 primary	1	100M docs, ~50GB indexed
Tier 2	6 (hot) + 2 (warm)	6 primary	1	500M docs, ~250GB indexed
Tier 3	12 (hot) + 4 (warm) + 2 (cold)	12 primary	1	3B docs, ~1.5TB indexed

7.3 Index Rollover Strategy

Instead of creating one giant index, monthly indices are created automatically:

```
Index alias: transactions-search → [ transactions-2026-01, transactions-2026-02, ... ]

When a month's index exceeds 50GB or 100M docs:
→ ILM automatically rolls over to a new index
→ Old index is optimized (force-merge to fewer segments)
→ After 3 months, moved to warm tier
→ After 12 months, moved to cold tier (frozen)
→ After 36 months, deleted (PG is the SoT)
```

8. MongoDB Scaling

8.1 MongoDB (Audit Logs) Scaling

Audit logs grow linearly with system usage. MongoDB Atlas auto-scales but here's the manual plan:

Tier	Cluster	Instance	Storage	Handles
Tier 1	3-node replica set	M40 (4 vCPU, 16GB)	500GB	1M audit events/day
Tier 2	3-node replica set	M50 (8 vCPU, 32GB)	2TB	5M audit events/day
Tier 3	Sharded cluster (3 shards × 3 nodes)	M60 (16 vCPU, 64GB)	10TB	25M audit events/day

Shard key for Tier 3: `{ userId: "hashed" }` — distributes writes evenly across shards.

8.2 TTL-Based Auto-Cleanup

```
// Automatically delete audit logs older than retention period
db.auditLogs.createIndex(
  { "timestamp": 1 },
  { expireAfterSeconds: 31536000 }  // 365 days
);

// Hot data stays in MongoDB. Cold data exported to S3 before TTL hits.
```

---

9. API Gateway Scaling

9.1 Gateway Scaling Strategy

The API Gateway (Kong / Spring Cloud Gateway) is stateless and scales horizontally:

```
┌──────────────────────────────────────────────────────────────┐
│  API GATEWAY SCALING                                          │
│                                                               │
│  Tier 1: 3 pods (HPA min=3, max=10)                           │
│  Tier 2: 5 pods (HPA min=5, max=20)                           │
│  Tier 3: 10 pods (HPA min=10, max=50)                         │
│                                                               │
│  Scale trigger: CPU > 60% OR requests/sec > 3000/pod          │
│                                                               │
│  The gateway is CPU-bound (JWT validation, rate limit checks) │
│  so CPU is the primary HPA metric.                            │
│                                                               │
│  All state (rate limits, JWT blacklist) is in Redis,           │
│  so gateway pods are fully stateless.                          │
└──────────────────────────────────────────────────────────────┘
```

9.2 Rate Limiting at Scale

Rate limiting uses Redis `INCR + EXPIRE` (sliding window). At 125K req/sec (Tier 3), Redis handles rate limit checks easily (Redis does 100K+ ops/sec per shard). No special scaling needed for rate limiting — Redis cluster handles it.

---

10. Microservice (Application Tier) Scaling

10.1 Per-Service HPA Configuration

Service	Min Pods	Max Pods	Scale Metric	CPU Target
Auth Service	3	15	CPU + login requests/sec	70%
Account Service	3	15	CPU	70%
Transaction Service	5	30	CPU + txn/sec	60% (most critical)
Payment Service	3	15	CPU + pending payments	70%
Loan Service	2	10	CPU	70%
Notification Service	3	20	Kafka consumer lag	Lag < 5000
Fraud Detection	2	10	CPU + inference latency	70%
Reporting Service	2	8	CPU	70%
Scheduler Service	2	2	N/A (fixed — Quartz leader election)	N/A
ES Index Consumer	3	12	Kafka consumer lag	Lag < 5000

10.2 Pod Anti-Affinity

```yaml
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
    - weight: 100
      podAffinityTerm:
        labelSelector:
          matchLabels:
            app: transaction-service
        topologyKey: topology.kubernetes.io/zone
```

Spreads pods across AZs. If AZ-1 goes down, pods in AZ-2 and AZ-3 continue serving.

10.3 Node Pool Strategy

```
┌──────────────────────────────────────────────────────────────────┐
│  KUBERNETES NODE POOLS                                            │
│                                                                   │
│  Pool: general-purpose                                            │
│  Instance: c6g.2xlarge (8 vCPU, 16GB)                            │
│  Min: 6 nodes | Max: 30 nodes                                     │
│  Services: All stateless microservices + BFF                      │
│  Autoscaler: Cluster Autoscaler (scale nodes when pods pending)│
│                                                                   │
│  Pool: memory-optimized                                           │
│  Instance: r6g.xlarge (4 vCPU, 32GB)                              │
│  Min: 3 nodes | Max: 10 nodes                                     │
│  Services: Kafka brokers, Redis (if self-hosted), ES nodes        │
│  Autoscaler: Manual (stateful workloads)                          │
│                                                                   │
│  Pool: spot-instances (cost optimization)                         │
│  Instance: c6g.2xlarge (8 vCPU, 16GB) - SPOT                      │
│  Min: 0 nodes | Max: 20 nodes                                     │
│  Services: Report generation, analytics, batch jobs               │
│  Autoscaler: Cluster Autoscaler                                   │
└──────────────────────────────────────────────────────────────────┘
```

---

11. End-to-End Scaling Scenario: 10x Traffic

Scenario: Diwali sale + salary day. Traffic spikes 10x for 4 hours.

```
Normal:  5,000 API req/sec, 500 txn/sec
Spike:   50,000 API req/sec, 5,000 txn/sec

┌──────────────────────────────────────────────────────────────────┐
│  SCALING TIMELINE                                                 │
│                                                                  │
│  T-24h:   Pre-scale (scheduled): set HPA min for critical        │
│           services to 3x current. Warm up PG connection pools.   │
│                                                                  │
│  T+0:     Traffic starts rising.                                  │
│                                                                  │
│  T+2min:  HPA detects CPU > 70% on BFF, Txn Service.             │
│           Doubles pods: BFF 3→6, Txn 5→10.                       │
│                                                                  │
│  T+3min:  Cluster Autoscaler detects pending pods.                │
│           Launches 4 new EC2 nodes (1-2 min boot time).           │
│                                                                  │
│  T+5min:  New pods scheduled and ready.                           │
│           HPA continues scaling: BFF 6→12, Txn 10→20.            │
│                                                                  │
│  T+10min: Kafka consumer lag rises (notifications).               │
│           Notification consumer scaled from 3→6 pods.             │
│                                                                  │
│  T+15min: System stabilized at:                                   │
│           • BFF: 15 pods (from 3)                                 │
│           • Txn Service: 20 pods (from 5)                         │
│           • Auth Service: 10 pods (from 3)                        │
│           • Notification: 6 pods (from 3)                         │
│           • Kafka: no change (12 partitions sufficient)            │
│           • Redis: no change (memory headroom sufficient)          │
│           • PG: read replicas absorb filter load                   │
│                                                                  │
│  T+4h:    Traffic subsides.                                       │
│                                                                  │
│  T+4h 5min: HPA starts scale-down (5-min stabilization window).  │
│             Removes 25% of pods every 2 min.                      │
│                                                                  │
│  T+4h 30min: Back to normal pod counts.                           │
│              Cluster Autoscaler drains and terminates extra nodes.│
└──────────────────────────────────────────────────────────────────┘
```

12. Scaling Bottleneck Analysis

12.1 What Breaks First?

```
Load increases →

1st bottleneck:  BFF/API Gateway (CPU-bound, JWT validation)
   Fix: HPA scales pods in 30-60 seconds ✓

2nd bottleneck:  PG write primary (connection saturation)
   Fix: PgBouncer absorbs connection spikes. Optimize slow queries. ✓

3rd bottleneck:  PG read replicas (filter query load)
   Fix: Add more replicas. Route to specific replicas by query type. ✓

4th bottleneck:  Kafka consumer lag (notification/audit consumers)
   Fix: Scale consumers (up to partition count). Add partitions if needed. ✓

5th bottleneck:  Redis memory (more sessions + cached data)
   Fix: Online resharding (add shard). Typically 10-30 min. ✓

HARD CEILING:    PG write primary throughput
   Single-primary PostgreSQL maxes out at ~20-50K writes/sec
   (depending on transaction complexity and locking).
   Beyond this → Citus sharding or architecture redesign.
```

12.2 Pre-emptive Actions to Avoid Bottlenecks

Bottleneck	Pre-emptive Action	When
PG write contention	Reduce lock duration (smaller transactions), batch writes	Before Tier 2
Kafka partition limit	Plan partition count for 3-year growth. Increasing partitions is easy; decreasing is impossible.	At topic creation
Redis memory ceiling	Use shorter TTLs aggressively. Archive old sessions. Monitor evictions.	Monthly review
ES shard count	Use ILM with monthly rollover. Avoid creating too many small shards (1 shard = 1 Lucene index = overhead).	At index template creation
K8s node capacity	Set cluster autoscaler with generous max. Pre-warm spot instance pools.	At cluster creation

13. Cost of Scaling

13.1 Infrastructure Cost by Tier

Component	Tier 1 (500K users)	Tier 2 (2M users)	Tier 3 (10M users)
EKS Nodes	6 × c6g.2xlarge = $2,400/mo	15 × c6g.2xlarge = $6,000/mo	30 × c6g.2xlarge = $12,000/mo
PostgreSQL (RDS)	1P + 1S + 2RR = $3,200/mo	1P + 1S + 4RR = $5,600/mo	1P(8xl) + 1S + 6RR = $14,400/mo
Redis	3M + 3R = $2,400/mo	6M + 6R = $4,800/mo	9M + 9R = $7,200/mo
Kafka (MSK)	3 brokers = $1,800/mo	5 brokers = $3,000/mo	9 brokers = $5,400/mo
Elasticsearch	3 data + 2 master = $2,800/mo	6 data + 2 warm = $5,600/mo	12 data + 4 warm = $11,200/mo
MongoDB Atlas	M40 × 3 = $1,800/mo	M50 × 3 = $3,200/mo	Sharded = $9,600/mo
Other (S3, CDN, monitoring)	~$1,500/mo	~$3,000/mo	~$8,000/mo
TOTAL	**~$15,900/mo**	**~$31,200/mo**	**~$67,800/mo**
Cost per user	**$0.032/mo**	**$0.016/mo**	**$0.007/mo**

Cost per user decreases as we scale — economies of scale from shared infrastructure (Kafka, Redis, monitoring amortized across more users).

13.2 Cost Optimization Strategies

Strategy	Savings	Detail
Spot instances for non-critical workloads	60-70% on compute	Reports, analytics, batch jobs run on spot. Interruptions are acceptable.
Reserved instances for baseline capacity	30-40% on RDS/ElastiCache	1-year reserved for primary PG, Redis masters, Kafka brokers.
ILM for Elasticsearch	40-50% on ES storage	Warm/cold tiers use cheaper storage. Only hot data on SSD.
Right-sizing after load testing	10-20% overall	Quarterly review: downsize over-provisioned instances.
PgBouncer	Avoid PG instance upgrade	Connection multiplexing avoids upgrading PG just for connection limits.
Aggressive TTLs	Delay Redis scaling	Shorter TTLs = less memory = delay adding Redis shards.

14. Scaling Runbook

14.1 Quick Reference: "I Need to Scale X"

What to Scale	Action	Time to Effect	Risk
BFF / any stateless service	Increase HPA min or wait for auto-scale	30-60 seconds	Low (stateless)
Kafka consumers	Increase pod replicas (up to partition count)	30 seconds (rebalance)	Low
Kafka partitions	`kafka-topics.sh --alter --partitions N`	Seconds (but irreversible)	Medium (rebalancing)
Kafka brokers	Add broker nodes to cluster	10-30 min (data rebalance)	Medium
Redis memory	Online resharding (add shard)	10-60 min	Low (online)
PG read capacity	Add read replica	15-30 min (initial sync)	Low
PG write capacity	Vertical scale (modify RDS instance)	5-15 min (brief downtime)	Medium
PG storage	Increase EBS volume	0 downtime, minutes	Low
ES query capacity	Add replica shards	Minutes (shard allocation)	Low
ES data capacity	Add data nodes	10-30 min (shard rebalance)	Medium
ES indexing speed	Add data nodes + increase refresh interval	Minutes	Low
K8s node capacity	Cluster Autoscaler (automatic)	2-5 min (EC2 launch)	Low

14.2 Scaling Checklist (Before Production Launch)

- [ ] HPA configured for all stateless services
- [ ] Cluster Autoscaler enabled with appropriate min/max node counts
- [ ] PgBouncer deployed in front of all PostgreSQL connections
- [ ] Read replicas configured and read-only queries routed to them
- [ ] Kafka partition counts set for 3-year growth projection
- [ ] Redis cluster deployed with multi-AZ replication

- [ ] Elasticsearch ILM policies configured (hot/warm/cold)
- [ ] Load test completed at 10x expected traffic (Gatling)
- [ ] Monitoring dashboards for all scaling metrics (Grafana)
- [ ] Alerts configured for scaling triggers (Prometheus Alertmanager)
- [ ] DR region provisioned (warm standby)
- [ ] Runbook reviewed and accessible to on-call team

End of Document — System Scaling Strategy v1.0